

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
Jnana Sangama, Belgaum-590018



Innovation Design Thinking

on

“ClinicaDiff – AI Powered Smart Healthcare Assistant”

Submitted in partial fulfillment of the requirements for the **Second Semester of the Bachelor of Engineering Degree**, towards the completion of the IDP

Project under the **Innovation & Design Thinking Laboratory**, Department ECE

by

SN	Name	USN
1	Ashwini D P	1CR25EC029
2	Avnish Kumar	1CR25EC030
3	Bhavesh Damani R	1CR25CS029
4	Bhavya Shree	1CR25CS030

Under the Guidance of Dr Sachin Associate Professor, Dept. of ECE



CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

DEPARTMENT OF ECE



CERTIFICATE

This is to certify that the File Structures IDP project entitled “Smart Health Monitoring Device” has been successfully carried out by Ashwini D P (1CR25EC029), Avnish Kumar (1CR25EC030), Bhavesh Damani R (1CR25CS029) and Bhavya Shree (1CR25CS030), bonafide students of **CMR Institute of Technology**.

The project is submitted in partial fulfillment of the requirements for the Second Semester of the Bachelor of Engineering Degree, towards the completion of the IDP project under the **Innovation & Design Thinking Laboratory, Department of ECE**

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures IDP project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Dr Sachin

**Associate Professor
Dept. of ECE , CMRIT**

Signature of HOD

Dr. Raveesha K H

**HoD,
Dept. of Physics, CMRIT**

External Viva

Name of the examiners

- 1.
- 2.

Signature with date

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H**, HoD, Dept. of Physics, CMRIT for his valuable guidance and support.

I am especially grateful to my internal guide, **Dr. Sachin**, Department of ECE, for his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

Ashwini D P	(1CR25EC029)
Avnish Kumar	(1CR25EC030)
Bhavesh Damani R	(1CR25CS029)
Bhavya Shree	(1CR25CS030)

TABLE OF CONTENTS

ACKNOWLEDGEMENT	I
ABSTRACT	II
LIST OF FIGURES	III
1. INTRODUCTION	7
1.1 Brief History of File Structures	7
1.2 Primary Goals for Design of file structures	7
2. ABOUT THE PROJECT	8
2.1 Problem Statement	8
2.2 Objective of the Project	8
2.3 Proposed Solution	8
2.4 Advantages of Proposed solution	8
3. DESIGN	9
3.1 Description of approach to solve the problem	9
3.2 Steps to implement FIFO in virtual memory management	9
4. CODE SNIPPETS AND DESCRIPTION	10-16
4.1 Generalized code for FIFO	10
5. RESULT AND ANALYSIS	17-19
5.1 Result	17
5.2 Screenshots	17
6. CONCLUSION	20
7. REFERENCES	21

LIST OF FIGURES

SL. No	Description	Page No
1	Dashboard	17
2	Dashboard	18
3	Domain Selection	18
4	Patient Condition Report	19

ABSTRACT

Healthcare systems require fast, accurate, and efficient decision-making to provide better patient care and reduce medical errors. Doctors often face challenges in analyzing large amounts of patient information, identifying diseases, and verifying medication compatibility within limited time. To address these challenges, ClinicaDiff is developed as an AI-powered smart healthcare assistant.

ClinicaDiff uses Artificial Intelligence and Machine Learning techniques to support doctors in real-time diagnosis, drug interaction analysis, and effective patient management. The system collects patient data such as symptoms, medical history, and prescribed medications, then processes the information to generate diagnostic suggestions and detect possible medication risks.

The platform also improves communication between doctors and patients through an easy-to-use interface. By automating healthcare analysis and supporting clinical decisions, ClinicaDiff helps reduce human errors, save time, improve patient safety, and enhance overall healthcare efficiency. This project demonstrates how intelligent healthcare solutions can contribute to the future of modern medical systems.

1. INTRODUCTION

1.1 Brief History of Healthcare Intelligence Systems

Healthcare has traditionally depended on doctors manually analyzing patient symptoms, medical history, and treatment records to make clinical decisions. Although effective, this process often requires more time and may lead to errors due to increasing patient data and workload. With advancements in digital technologies, healthcare systems evolved from paper-based records to electronic healthcare management systems.

Artificial Intelligence (AI) and Machine Learning further transformed healthcare by enabling disease prediction, smart diagnosis support, patient monitoring, and medication analysis. ClinicaDiff is developed based on these advancements to provide an AI-powered healthcare platform that supports doctors with diagnosis assistance, drug interaction analysis, and seamless patient communication.

1.2 Primary Goals for Design of ClinicaDiff

The primary goal of ClinicaDiff is to design an intelligent healthcare support system that improves decision-making and reduces medical errors. The system aims to:

- Provide real-time diagnosis assistance.
- Detect harmful drug interactions.
- Improve communication between doctors and patients.
- Increase healthcare efficiency and accuracy.
- Reduce time spent on manual medical analysis.

2. ABOUT THE PROJECT

2.1 Problem Statement

Modern healthcare systems face challenges such as delayed diagnosis, medication errors, and difficulty in managing large volumes of patient data. Manual verification of symptoms and medicines consumes time and may affect treatment quality. Therefore, an AI-based solution is required to support doctors and improve patient care.

2.2 Objective of the Project

The objective of ClinicaDiff is to build an AI-powered healthcare assistant that supports medical professionals by providing diagnostic suggestions and medication analysis. The project aims to improve patient safety, reduce errors, and enable efficient communication between doctors and patients.

2.3 Proposed Solution

ClinicaDiff proposes an AI-based healthcare platform that processes patient information such as symptoms, medical history, and medication details. Using machine learning algorithms, the system predicts possible conditions and identifies unsafe drug combinations. The platform provides quick recommendations to support clinical decisions.

2.4 Advantages of Proposed Solution

- Faster and more accurate diagnosis support
- Drug interaction detection
- Reduced medication errors
- Better doctor-patient collaboration
- Improved patient safety
- Time-saving healthcare automation

3. DESIGN

3.1 Description of Approach to Solve the Problem

The project follows an AI-based approach to healthcare assistance. Patient data is collected through the interface and processed using machine learning techniques. The diagnosis module predicts possible diseases based on symptoms, while the drug interaction module checks medication compatibility. Results are displayed to doctors through an easy-to-use dashboard.

3.2 Steps to Implement ClinicaDiff

1. Requirement gathering and analysis
2. Collection of healthcare datasets
3. Data preprocessing and cleaning
4. Development of AI prediction models
5. Drug interaction module integration
6. Database development
7. User interface implementation
8. System testing and deployment

4. CODE SNIPPETS AND DESCRIPTION

4.1 Generalized Code Description

The ClinicaDiff system is developed using Python and AI technologies. The frontend allows users to input patient details and view results. The backend processes the data and interacts with machine learning models.

Main modules include:

- Patient Data Input Module
- Diagnosis Prediction Module
- Drug Interaction Analysis Module
- Database Management Module
- Communication Interface Module

The system architecture is designed to be modular and scalable for future improvements.

Code:

```
import re
with open('app.py', 'r', encoding='utf-8') as f:
    content = f.read()
    new_ocr = '@app.route('/ocr', methods=['POST'])
    def ocr_prescription():
        """OCR endpoint — accepts image file OR base64 JSON."""
        global ocr_reader
        try:
            temp_dir = os.path.join(os.path.dirname(__file__), 'temp')
            os.makedirs(temp_dir, exist_ok=True)
            temp_path = os.path.join(temp_dir, '_ocr_input.png')

            # Accept multipart file upload
            if 'image' in request.files:
                image_file = request.files['image']
                image_file.save(temp_path)
            # Accept base64 JSON
            elif request.is_json and request.json.get('image'):
                b64 = request.json['image']
                b64 = re.sub(r'^data:image/\w+;base64,', '', b64)
                img_bytes = base64.b64decode(b64)
```

```
with open(temp_path, 'wb') as f:
    f.write(img_bytes)
else:
    return jsonify({"status": "error", "message": "No image provided. Send as file upload
or base64 JSON."}), 400

# Check file exists and has content
if not os.path.exists(temp_path) or os.path.getsize(temp_path) < 100:
    return jsonify({"status": "error", "message": "Image file is empty or corrupt"}), 400

print(f"[OCR] Image saved: {os.path.getsize(temp_path)} bytes")

# -----
# Step 1: EasyOCR Extraction
# -----
image_paths = preprocess_image(temp_path)
if isinstance(image_paths, str):
    image_paths = [image_paths]

if ocr_reader is None:
    ocr_reader = easyocr.Reader(['en'], gpu=False)

best_results = []
for p in image_paths:
    try:
        results = ocr_reader.readtext(p, detail=0, paragraph=True, text_threshold=0.5,
        low_text=0.3, width_ths=0.7)
        if len(results) > len(best_results):
            best_results = results
    except Exception as e:
        pass

    try:
        results_no_para = ocr_reader.readtext(temp_path, detail=0, paragraph=False,
        text_threshold=0.5, low_text=0.3)
        if len(results_no_para) > len(best_results):
            best_results = results_no_para
    except:
        pass
```

```
raw_text = "\\n".join(best_results) if best_results else ""
print(f"[OCR] Extracted {len(best_results)} text blocks, {len(raw_text)} chars")
```

```
# -----
```

```
# Step 2: Groq Structuring
```

```
# -----
```

```
if groq_client and raw_text.strip():
```

```
try:
```

```
print("[OCR] Attempting Groq extraction...")
```

```
safe_text = raw_text.replace("'", '\\\\').replace('\\n', ' ')
```

```
prompt = f"""
```

```
You are a medical AI assistant. Read this raw text extracted from a handwritten
medical prescription via OCR:
```

```
"{safe_text}"
```

Even if the text is messy or has OCR errors, try your absolute best to deduce the drug names, dosages, and patient info based on your vast medical knowledge.

Respond ONLY with a valid raw JSON object (NO markdown wrappers like ``json) with the following structure:

```
{
  "raw_text": "{safe_text}",
  "drugs_found": ["DrugName1", "DrugName2"],
  "medications": [
    {
      "drug_name": "Full drug name (brand name if visible)",
      "generic_name": "Generic/chemical name if known",
      "dosage": "e.g. 10mg, 500mg",
      "frequency": "e.g. Once daily, Twice daily, Three times daily",
      "duration": "e.g. 7 days, 1 month, As needed",
      "timing": "e.g. After meals, Before bed, Morning",
      "form": "e.g. Tablet, Capsule, Syrup, Gel, Injection",
      "instructions": "Any special instructions in simple patient-friendly language"
    }
  ],
  "patient_info": {
    "age": "Patient age if visible",
    "name": "Patient name if visible",
    "date": "Prescription date if visible",
    "doctor_name": "Doctor name if visible"
  }
}
```

```
}},  
"diagnosis": "Any diagnosis mentioned on the prescription",  
"special_notes": "Any additional notes or warnings on the prescription",  
"has_doctor_signature": true  
}}
```

IMPORTANT: For each medication, write the instructions in simple language a patient can understand.

"""

```
response = groq_client.chat.completions.create(  
model="llama-3.3-70b-versatile",  
messages=[{"role": "user", "content": prompt}],  
temperature=0.2,  
max_tokens=2048,  
)  
res_text = response.choices[0].message.content.strip()  
if res_text.startswith("```json"):  
res_text = res_text[7:-3]  
elif res_text.startswith("```"):  
res_text = res_text[3:-3]
```

```
data = json.loads(res_text.strip())
```

Trust Ledger Simulation: Cryptographic Verification

```
has_sig = data.get("has_doctor_signature", False)  
crypto_status = "VERIFIED (Cryptographic Ledger Sync)" if has_sig else "UNVERIFIED  
(No valid signature detected)"
```

Build a clean patient summary from structured data

```
meds = data.get("medications", [])  
summary_parts = ["[PRESCRIPTION SUMMARY]"]  
if meds:  
summary_parts.append("")  
for i, med in enumerate(meds, 1):  
line = f"{i}. {med.get('drug_name', 'Unknown')} {med.get('dosage', '')}"  
line += f" — {med.get('frequency', '')} for {med.get('duration', 'as directed')}"  
if med.get('instructions'):  
line += f". {med['instructions']}"  
summary_parts.append(line)  
patient_summary = "\\n".join(summary_parts)
```

```
# Cleanup
for f in os.listdir(temp_dir):
try: os.remove(os.path.join(temp_dir, f))
except: pass

return jsonify({
    "status": "success",
    "raw_text": data.get("raw_text", raw_text),
    "cleaned_text": data.get("raw_text", raw_text),
    "expanded_text": data.get("raw_text", raw_text),
    "drugs_found": data.get("drugs_found", []),
    "medications": meds,
    "patient_info": data.get("patient_info", {}),
    "diagnosis": data.get("diagnosis", ""),
    "special_notes": data.get("special_notes", ""),
    "abbreviations_found": [],
    "patient_summary": patient_summary,
    "text_blocks_count": len(best_results),
    "engine": "groq",
    "trust_ledger": {
        "signature_detected": has_sig,
        "verification_status": crypto_status,
        "timestamp": "Real-time"
    }
})
except Exception as e:
    print(f"[OCR] Groq Failed: {e}. Falling back to Rule-based.")

# -----
# Step 3: Fallback (If no API key or Groq fails)
# -----
# Clean
cleaned = re.sub(r"^[^\\x20-\\x7E\\n]+", "", raw_text)
cleaned = re.sub(r"[\\t]+", " ", cleaned)
lines = [ln.strip() for ln in cleaned.splitlines() if ln.strip()]
cleaned = "\\n".join(lines)

# Expand abbreviations
expanded = cleaned
```

```
for abbr, full in ABBREVIATIONS.items():
    expanded = re.sub(rf"\\b{abbr}\\b", f"{abbr} ({full})", expanded,
    flags=re.IGNORECASE)

# Find drugs
drugs_found = find_drugs_in_text(cleaned)

# Find abbreviations
abbrs_found = [
    {"abbr": abbr, "meaning": full}
    for abbr, full in ABBREVIATIONS.items()
    if re.search(rf"\\b{abbr}\\b", cleaned, re.IGNORECASE)
]

# Cleanup temp files
for f in os.listdir(temp_dir):
    try:
        os.remove(os.path.join(temp_dir, f))
    except:
        pass

# Generate patient-friendly summary
patient_summary = generate_patient_summary(cleaned, drugs_found, abbrs_found)
if not groq_client:
    patient_summary = "[WARNING] Add GROQ_API_KEY in backend/.env for highly
    accurate handwriting AI!\\n\\n" + patient_summary

return jsonify({
    "status": "success",
    "raw_text": raw_text,
    "cleaned_text": cleaned,
    "expanded_text": expanded,
    "drugs_found": drugs_found,
    "abbreviations_found": abbrs_found,
    "patient_summary": patient_summary,
    "text_blocks_count": len(best_results),
    "engine": "easyocr"
})

except Exception as e:
```

```
import traceback
traceback.print_exc()
return jsonify({"status": "error", "message": str(e)}), 500'''

start_idx = content.find("@app.route('/ocr', methods=['POST'])")
end_idx = content.find("@app.route('/calculate-hereditary-risk', methods=['POST'])")

if start_idx != -1 and end_idx != -1:
    new_content = content[:start_idx] + new_ocr + "\n\n\n" + content[end_idx:]
    with open('app.py', 'w', encoding='utf-8') as f:
        f.write(new_content)
    print("Success")
else:
    print("Failed to find endpoints")
```


5. RESULT AND ANALYSIS

5.1 Result

ClinicaDiff successfully provides real-time diagnosis assistance and medication interaction analysis. The AI model generates healthcare recommendations based on patient input and helps doctors make informed decisions. The platform improves workflow efficiency and supports safer treatment planning.

5.2 Screenshots

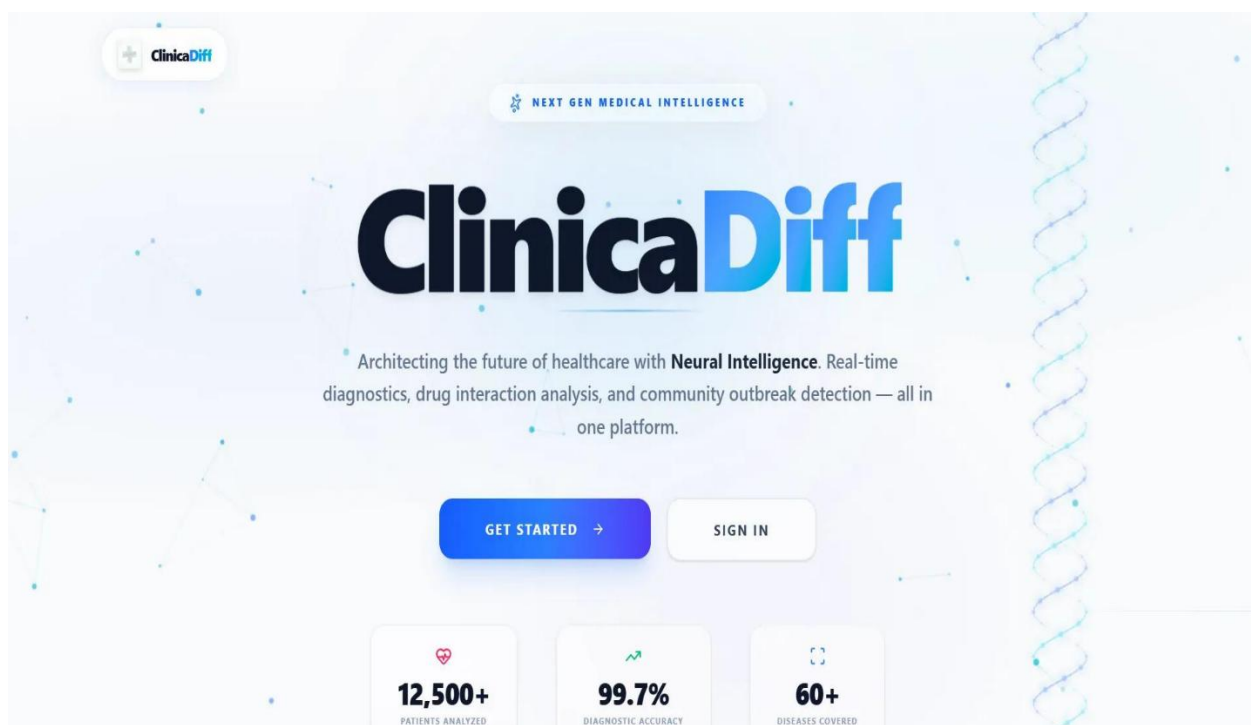


Fig.1

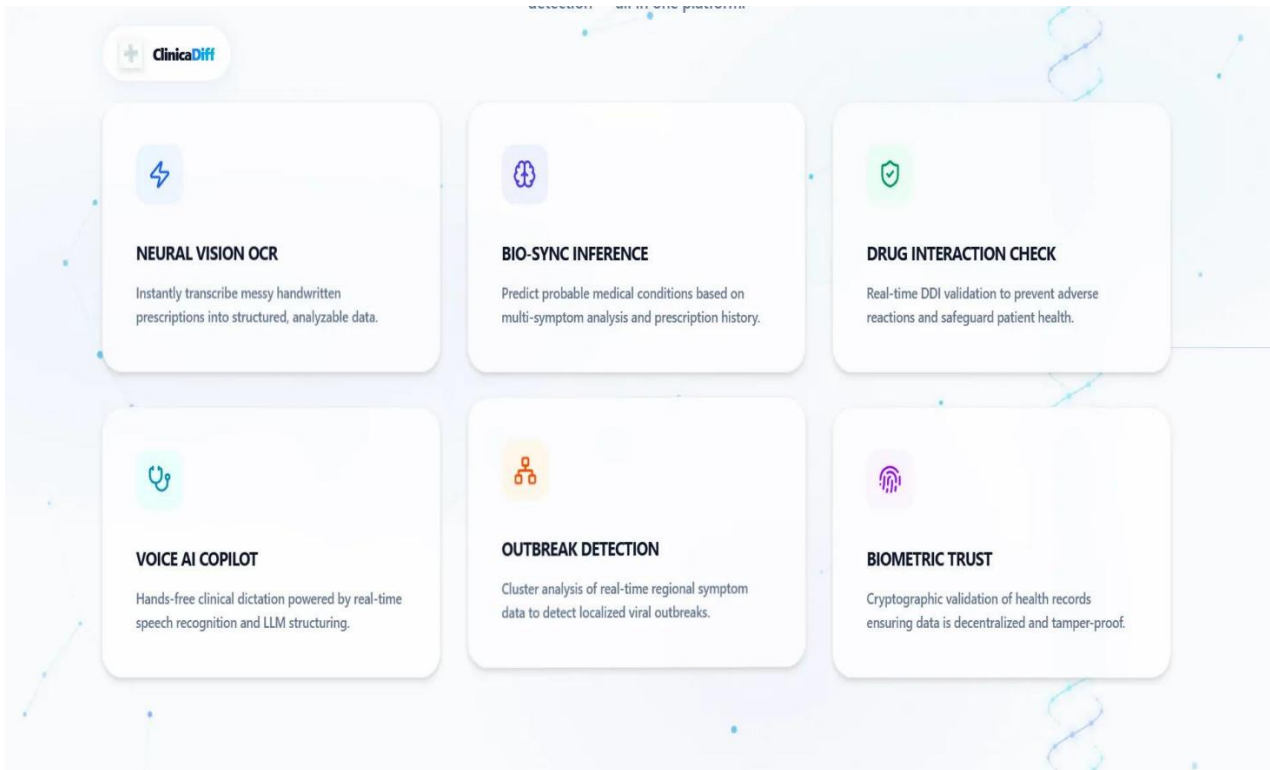


Fig.2

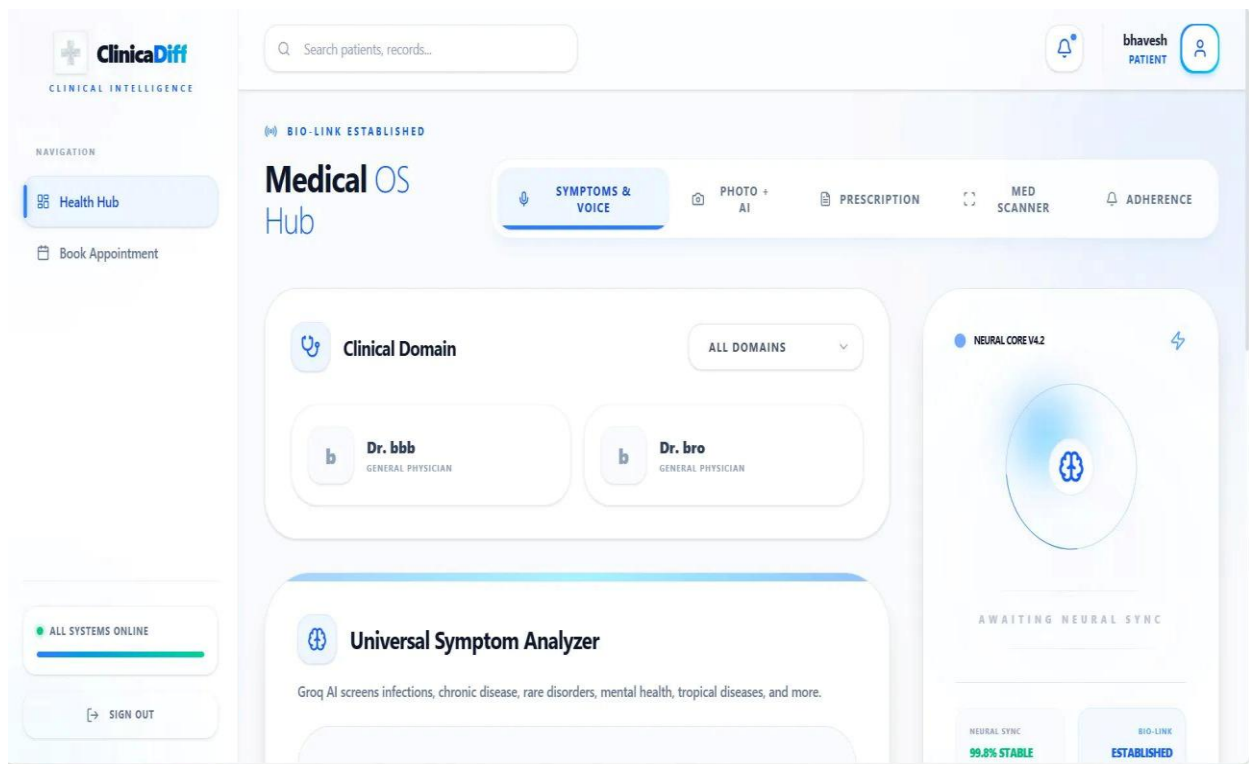


Fig.3

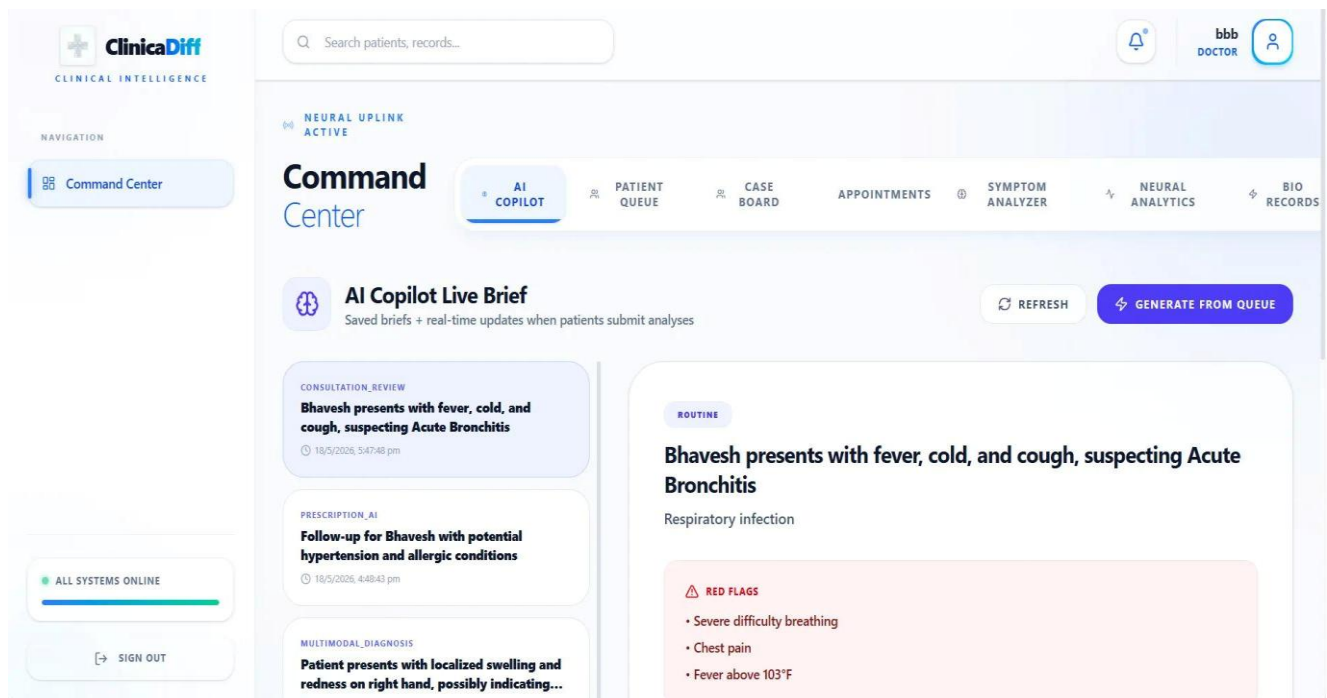


Fig.4

6. CONCLUSION

ClinicaDiff is an AI-powered healthcare assistance platform developed to improve medical decision-making and patient care. The system provides diagnosis support, medication safety analysis, and effective communication between doctors and patients. By integrating AI into healthcare processes, ClinicaDiff reduces errors, saves time, and improves overall healthcare quality.

7. REFERENCES

- [1] D. R. Serrano et al., “Artificial Intelligence (AI) Applications in Drug Discovery and Healthcare,” *National Library of Medicine (PMC)*, 2024.
- [2] P. Sonaji, L. Subramanian and M. Rajesh, “Artificial Intelligence-Driven Drug Interaction Prediction,” *World Journal of Biology Pharmacy and Health Sciences*, vol. 17, no. 2, pp. 297–305, 2024.
- [3] Y. Lu et al., “Machine Learning Models for Drug–Drug Interaction Prediction,” *npj Digital Medicine*, 2026.
- [4] A. U. Rehman et al., “Role of Artificial Intelligence in Revolutionizing Drug Discovery,” *Drug Discovery Today*, 2025.
- [5] European Medicines Agency (EMA), “Review of Artificial Intelligence and Machine Learning Applications in Medicines Lifecycle,” Technical Report, 2025.